Agenda:

1. Check Bit
2. Set Bit
3. Flip Bit
4. Unset Bit
5. Number of Subarrays with OR = 0
6. Max AND Pair

Q1: Given an integer **N** and a position **i** (0-indexed), **check** if **i-th** bit of **N** is **1**.

Example: N = 45 $\boxed{i = 2}$ Ans = 1

$\boxed{i = 4}$ Ans = 0

5	4	3	2	1	0
1	0	1	1	0	1

Left Shift

$1 << 2 =$

3	2	1	0
0	0	0	1

0100 = 4

$\boxed{1 << n = 2^n}$

$i = 2$

$i = 4$

	5	4	3	2	1	0
45 →	1	0	1	1	0	1
1 << 2 → &	0	0	0	1	0	0
	0	0	0	1	0	0

$= 1 << 2$

$= 2^2$

	5	4	3	2	1	0
45 →	1	0	1	1	0	1
1 << 4 → &	0	1	0	0	0	0
	0	0	0	0	0	0

$= 0$

$$\boxed{\begin{array}{l} N \,\&\, (1 << i) \rightarrow (1 << i) \Rightarrow i^{th} \text{ bit is 1 in N} \\ \qquad\qquad\qquad\quad\;\; 0 \Rightarrow i^{th} \text{ bit is 0 in N} \end{array}}$$

	5	4	3	2	1	0
N = 45	1	0	1	1	0	1
i = 3	0	0	0	1	0	1

$(N >> i) \,\&\, 1 = 1 \Rightarrow i^{th}$ bit is 1

$$\boxed{\begin{array}{l} (N >> i) \,\&\, 1 \rightarrow 1 \Rightarrow i^{th} \text{ bit is 1 in N} \\ \qquad\qquad\qquad\quad\;\; 0 \Rightarrow i^{th} \text{ bit is 0 in N} \end{array}}$$

AND → Check

Q2: Given an integer **N** and a position **i** (0-indexed), **set** the **i-th** bit of **N** to **1**.

If the **i-th** bit is already set (**= 1**), leave N unchanged.

Example: N = 45 $\boxed{i = 2}$ → Ans = 45

$\boxed{i = 4}$ → Ans = 61

5	4	3	2	1	0
1	0	1	1	0	1
1	1	1	1	0	1

$= 61$

$i = 4$

	5	4	3	2	1	0
N = 45	1	0	1	1	0	1
|						
1 << 4	0	1	0	0	0	0
	1	1	1	1	0	1

$\rightarrow 61$

$i = 3$

	5	4	3	2	1	0
N = 45	1	0	1	1	0	1
|						
1 << 3	0	0	1	0	0	0
	1	0	1	1	0	1

$\rightarrow 45$

$$\boxed{\begin{array}{l} \underbrace{N \mid (1 \ll i)}_{i^{th}\text{ bit is 1}} \rightarrow N \rightarrow i^{th}\text{ bit was 1} \\ \qquad\qquad\qquad\quad \rightarrow N + (1 \ll i) \rightarrow i^{th}\text{ bit was 0} \end{array}}$$

OR $\rightarrow$ Set

Q3: Given an integer **N** and a position **i** (0-indexed), **toggle** (**flip**) the **i-th** bit of **N**.

If the bit is **0**, change it to **1**. If it is **1**, change it to **0**.

Example: N = 45

	5	4	3	2	1	0
	1	0	1	1	0	1

$\boxed{i = 2} \rightarrow$ 1 0 1 0 0 1 $\rightarrow 41$

$\boxed{i = 4} \rightarrow$ 1 1 1 1 0 1 $\rightarrow 61$

$i = 4$

	5	4	3	2	1	0
N = 45	1	0	1	1	0	1
^						
1 << 4	0	1	0	0	0	0
	1	1	1	1	0	1

$\rightarrow 61$

$i = 3$

	5	4	3	2	1	0
N = 45	1	0	1	1	0	1
^						
1 << 3	0	0	1	0	0	0
	1	0	0	1	0	1

$\rightarrow 37$

XOR

Flip i^{th} bit

$$\boxed{\begin{array}{l} \underline{N \wedge (1 \ll i)} \rightarrow N - (1 \ll i),\ i^{th}\text{ bit is 1} \\ \qquad\qquad\qquad\quad \rightarrow N + (1 \ll i),\ i^{th}\text{ bit is 0} \end{array}}$$

Q4: Given an integer **N** and a position **i** (0-indexed), **unset** the **i-th** bit of **N**.

If the **i-th** bit is already unset (**= 0**), leave N unchanged.

Example: N = 45

$\boxed{i = 2}$ → 5 4 3 2 1 0: 1 0 1 0 0 1 = 41

$\boxed{i = 4}$ → 1 0 1 1 0 1 = 45

5 4 3 2 1 0
1 0 1 1 0 1

```
if (checkBit(N, i))        // (N & (1<<i) == (1<<i))
    flipBit(N, i)          // N = N ^ (1<<i)
```

AND + XOR

check	AND	N & (1<<i) → (1<<i) / 0
set	OR	N | (1<<i)
flip	XOR	N ^ (1<<i)
unset	AND + XOR	if (N & (1<<i) == (1<<i)) N = N ^ (1<<i)

Q5: You are given an array **A[ ]** consisting of **0**s and **1**s. Your task is to calculate the number of subarrays for which the **bitwise OR** of all the elements in the subarray is **0**.

Input

0 1 2 3

A = [1, 0, 0, 1]

Output

3

Explanation

Subarrays with OR = 0

index	Subarray
{1}	[0]
{2}	[0]
{1, 2}	[0, 0]

0 1 2 3 4

A = [1 0 1 0 0]

Ans = 4

(1,1) (3,3) (4,4) (3,4)

0 1 2 3 4 5 6 7 8 9 10 11 12

A = [1 0 0 1 1 0 0 0 1 0 1 1 1]

$\frac{2*(2+1)}{2} = 3$ $\frac{3*4}{2} = 6$ 1

Ans = 3+6+1 = 10

subarray with OR = 0 ⇒ all elements in subarray = 0

Subarrays of 2 length array

$= \frac{n*(n+1)}{2}$

```
ans = 0        crt = 0
for i → 0 to (N-1) {
    if (A[i] == 0)  crt++
    else {  ans += crt * (crt+1) / 2
            crt = 0
    }
}
ans += crt * (crt+1) / 2
```

A = [... 1 0 0 0]

crt = 3

return ans

TC = $\underline{O(N)}$ SC = $\underline{O(1)}$

Find # subarrays with OR = 1 . $\boxed{\forall i,\ A[i] = \{0, 1\}}$

Subarray OR → 1, → 0

Total # subarrays = N*(N+1) /2

− <u>#subarrays with OR = 0</u>

= <u># subarrays with OR = 1</u>

Max AND Pair

Q7: Given N array elements, choose two indices(i, j) such that (**i != j**) and (**A[i] & A[j]**) is **maximum**.

$A[i] <= 10^9$

0 1 2 3 4

Example: A = [5, 4, 6, 8, 5]

Ans = **5** (A[0] & A[4])

5 → 0 1 0 1

4 → 0 1 0 0

6 → 0 1 1 0

8 → 1 0 0 0

i	j	A[i] & A[j]
0	1	4
0	2	4
0	3	0
0	4	5 (Ans)
1	2	4
1	3	0
1	4	4
2	3	0
2	4	4
3	4	0

$\overbrace{3 \quad \underbrace{4}_{0}}^{0} \quad 11$

$3 \,\&\, 11 = \underline{3}$

MSB ↑ 0 1 1 1

MSB ↑ 1 0 0 0

7 < 8

bits → $\boxed{29 \longrightarrow 0}$

$2^{30} > 10^9$

0	1	2	3	4	5	6

A = [26, 13, 23, 28, 27, 7, 25]

	4	3	2	1	0
26:	1	1	0	1	0
~~13~~:	0	1	1	0	1
~~23~~:	1	0	1	1	1
~~28~~:	1	1	1	0	0
27:	1	1	0	1	1
~~07~~:	0	0	1	1	1
~~25~~:	1	1	0	0	1

1 1 0 1 0 → 26

$1 <= A[i] <= 10^9$

```
ans = 0
for b → 29 to 0 {
    cnt = 0
    for i → 0 to (N-1) {
        if ( A[i] & (1<<b) > 0)
            cnt++
    }
    if (cnt >= 2) {
        ans = ans | (1<<b)
        for i → 0 to (N-1) {
            if ((A[i] & (1<<b)) == 0)
                A[i] = 0
        }
    }
}
return ans
```

cnt += (A[i] >> b) & 1 → {0, 1}

TC = O(N * 30) = O(N)

SC = O(1)

```
 int ans = 0;

for(int i = 31; i >= 0; i--)
{
    //count no. of set bits at ith index
    int count = 0;

    for(int j = 0; j < n; j++) {
        if(arr[j] & (1 << i))
            count++;
    }

    //set that bit in ans if count >=2
    if(count >= 2) {
        ans = ans | (1 << i);

        //set all numbers which have 0 bit at this position to 0
        for(int j = 0; j < n; j++) {
            if(arr[j] & (1 << i) == 0)
                arr[j] = 0;
        }

    }
}

print(ans);

//The numbers which cannot be choosen to form a pair have been made zero
```

What will be the outpout for following code snippet?

```
int x = 42;
int i = 3;
int res = x & (1 << i) == 0 ? 0 : 1;
System.out.println(res);
```

F

8

42 & false → error

Note: In Java, precedence order (for this question):

<<, ==, &, ?: (<< has highest priority).

What will be the outpout for following code snippet?

```
int x = 20;
x = x ^ (1 << 2);

x = x | (1 << 1);

if ((x & (1 << 4)) != 0)
    x = x ^ (1 << 4);

System.out.println(x);
```

4 3 2 1 0

1 0 1 0 0

^ 0 0 1 0 0

1 0 0 0 0

| 0 0 0 1 0

1 0 0 1 0

^ 1 0 0 0 0

0 0 0 1 0 → 2

What will be the outpout for following code snippet?

```
int x = 27;
int i = 3;
x = x ^ (1 << i);
x = x ^ (1 << (i - 1));
System.out.println(x);
```

4 3 2 1 0

27 → 1 1 0 1 1

1<<3 → ^ 0 1 0 0 0

1 0 0 1 1

1<<2 → ^ 0 0 1 0 0

1 0 1 1 1 → 23